Hub recovery and self-healing formation: my part

Owner: Vitalii · Add-on to "Hackathon plan: resilient rescue drone swarm"

TL;DR for the team

- **Your parts don't change.** Persons 1, 2 and 3 keep exactly what's in the main plan. This document adds two pieces that sit on top of your work.
- **Piece A, hub recovery:** every drone keeps a copy of what the hub knows. If the hub fails, nothing is lost and another drone can pick up the hub's job.
- **Piece B, gecko demo:** a Growing Neural Cellular Automaton (the "gecko" network) trained on the **relay chain** shape. It's shown as a side panel that regrows the chain when a drone fails. This is visual and for the pitch, and the main demo doesn't depend on it.
- **What I need from you:** two small hooks, listed under "Touchpoints with your parts".

Piece A: hub recovery

What problem it solves

The main plan already keeps the swarm searching without the hub. What's still missing is what happens to the **information** the hub was holding. Without this piece, a hub failure can lose survivor reports and the list of failed drones.

Demo metric: **"Survivor reports lost when the hub failed: 0"**, with the new hub highlighted on the map.

What the hub knows, and how each piece survives

Recomputed by any drone (no backup needed)	Copied to every drone
Sector split (Person 2's deterministic split)	Roster: per drone `(version, alive)`
Each drone's direction / target	Survivor reports + `delivered_to_base` flag
Hop-count gradient to base (Person 2)	Visited-cell set (Person 3's map merge)
	`hub_id`, `hub_epoch`

The copied state is only a few kilobytes. Every drone keeps a full copy and attaches it to every message it sends to a neighbour. That means no dedicated backup drone.

Merge rules (never conflict)

Field	Rule
Survivors	union; `delivered` = OR
Visited cells	union (same as Person 3's map merge)
Roster	per drone, higher `version` wins. Heard alive = bump own version; marked failed = others bump it. A drone hidden behind a ridge can come back.

Field	Rule
Hub	higher `hub_epoch` wins, lowest ID breaks ties

If a blackout splits the swarm, each half may pick its own hub. When the halves reconnect, the copies merge and the hub with the higher epoch keeps the role. While split, the halves' sector splits may overlap. Overlap is fine, gaps are not.

Taking over as hub

- Each drone records the last tick it heard from the hub, directly or through other drones. After **T = 10 ticks** of silence, the hub is suspect.
- The lowest live ID that a drone can reach becomes hub with `epoch + 1`.
- If the old hub comes back, its epoch is lower, so it steps down and merges.
- This fits the main plan's "hub is optional" rule. The swarm keeps searching the whole time. The new hub only brings back the optimising.

Piece B: gecko demo (self-healing relay chain)

Why it fits the main plan

The main plan's verdict was to give the network a real job: "a relay chain back to base that heals itself when a drone fails". This piece does exactly that. - **A relay chain is a fixed shape, so it can be trained**, like the gecko. - **Each drone uses only its neighbours and the same small learned rule.** When a relay fails, its neighbours slide in to close the gap, like a defender moving into the middle after a red card.

What I build

1. **Train the network.** Use `vendor/self-organising-systems/notebooks/growing_ca.ipynb` on a 16–40 px picture of a relay chain (base → far edge) instead of the gecko emoji. Train on a Colab GPU in Regenerating mode, which trains it to repair damage.
2. **Export the trained model for the browser.** The notebook already exports to JSON for TensorFlow.js.
3. **Side panel in the UI.** It shows the chain growing from one seed cell. When a drone fails in the simulation, the panel cuts the same hole, and the chain regrows live.

Honest limits (for judge questions)

- **It needs neighbours to talk every step.** It works while drones can see each other. Cut-off drones fall back to the main plan's "fully cut off" mode and keep sweeping alone.
- **It can't create drones.** The gecko regrows by creating new pixels. Here the picture means *where drones should be*, and the surviving drones spread out to fill it.
- **It's a visual proof of concept on a pixel grid.** Person 3's hand-coded relay chain is what the simulation actually runs. Moving the network onto the real drone network is future work.

Pitch line: *"Same idea as the self-repairing gecko: each drone follows one tiny learned rule, and the chain repairs itself without a leader."*

Touchpoints with your parts

Who	What I need	Effort for you
All, interface hour	`drone.step(local_observations) -> action` gets two extra fields: `local_observations.messages` (neighbours' state copies) and `action.outgoing_state` (this drone's copy)	Agree the format, ~10 min
Person 1 (sim/UI)	Deliver `messages` between drones in radio range (you already compute who can talk). A slot in the UI for the gecko panel. Highlight the current hub. Survivor-loss counter.	Small
Person 2 (coordination)	Your sector split takes the roster from my `SwarmState` instead of a fixed list. Your hub reassignment runs on whichever drone is hub now.	Small
Person 3 (drone policy)	Your visited-cell map merge becomes my `visited` field, so we don't build it twice. Your hand-coded relay chain stays yours. My network only mirrors its shape in the panel.	None / shared code

My build order and checkpoints

1. **First hour:** agree the message fields with everyone. Write `SwarmState` + `merge()` with tests (order doesn't matter, merging twice is safe, ridge-split scenario).
2. Copies passed on every neighbour contact. Suspect timer. Hub takeover.
3. **Demo-ready A:** fail the hub → new hub highlighted → survivor-loss counter stays 0.
4. Start the network training on Colab in the background, in parallel with step 2.
5. **Demo-ready B:** gecko panel in the UI, linked to drone failures.
6. Rehearse my two moments in the demo running order: step 2 (judge fails the hub) and step 5 (lift the blackout, copies merge).

If time runs out: Piece A is required. Piece B can fall back to a recorded video of the panel regrowing.